

TÉCNICA DE PROGRAMACIÓN ORIENTADA A OBJETOS
226513.5612(Remoto)

TEMA:

GUIA DE PRIMEROS PASOS GIT

Docente:

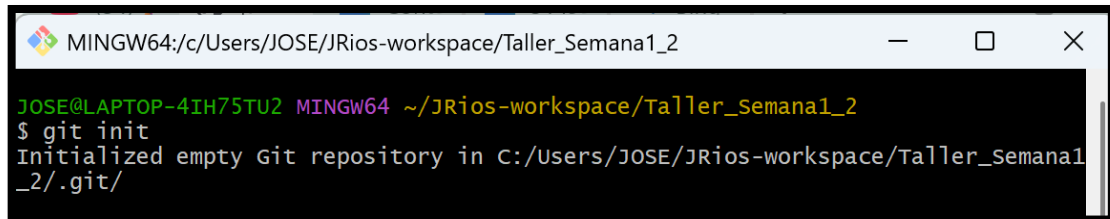
MARTIN EDUARDO TORRES RODRIGUEZ

Integrantes Grupo N.º 10:

José Andrés Abanto Trigos	N00468359
Aleida Mercedes Guillen Callapani	N00475469
Franklin Miguel Lopez Rodriguez	N00495538
Jose Luis Rios Huerto	N00422691
Renato Alexander Zavala Palomino	N00522496

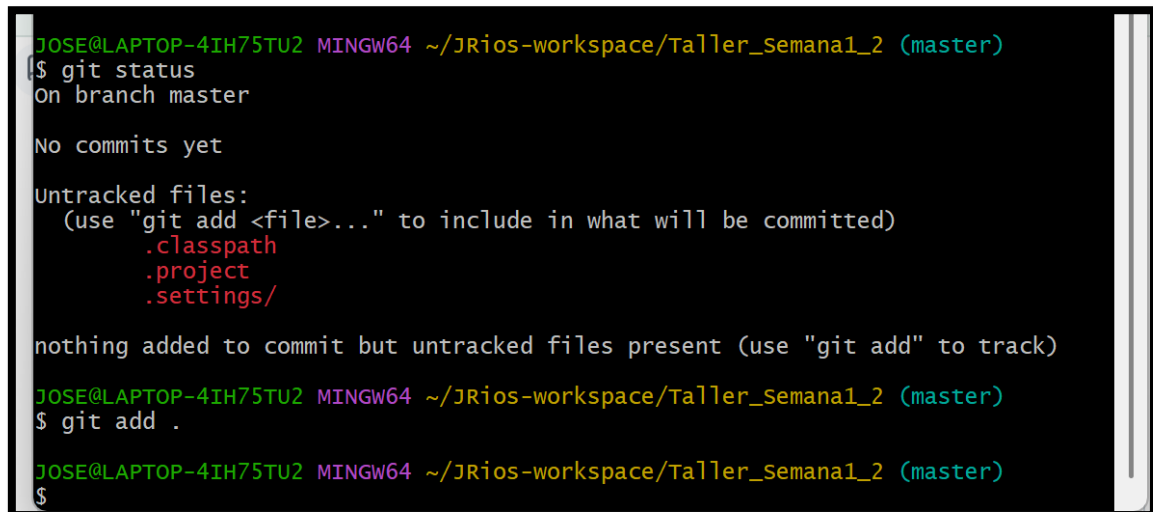
GUÍA DE PRIMERO PASOS CON GIT

1. SE INICIA CREANDO LA CARPETA LOCAL DENTRO DE WORKSPACE DE ECLIPSE CON EL COMANDO **GIT INIT**.



```
MINGW64:/c/Users/JOSE/JRios-workspace/Taller_Semana1_2
JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2
$ git init
Initialized empty Git repository in C:/Users/JOSE/JRios-workspace/Taller_Semana1_2/.git/
```

2. CON **GIT STATUS** PODEMOS VER QUÉ ESTÁ PASANDO EN NUESTRO PROYECTO:
NOS DICE SI HAY ARCHIVOS NUEVOS, MODIFICADOS, ELIMINADOS O SI TODO ESTÁ GUARDADO.



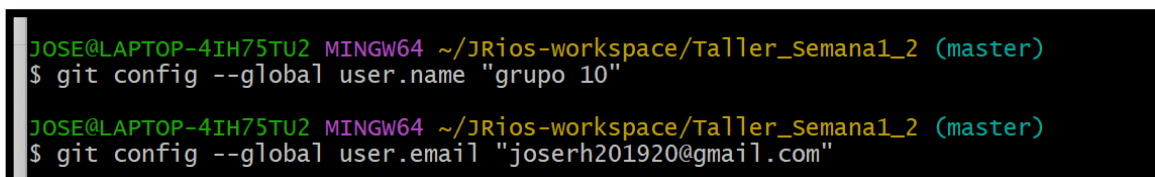
```
JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .classpath
        .project
        .settings/

nothing added to commit but untracked files present (use "git add" to track)
JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$ git add .
JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$
```

3. **GIT CONFIG --GLOBAL**: SIRVE PARA CONFIGURAR DATOS GENERALES DE GIT EN TU COMPUTADORA, COMO TU NOMBRE DE USUARIO Y CORREO, PARA QUE SE USEN EN TODOS TUS REPOSITORIOS.



```
JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$ git config --global user.name "grupo 10"
JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$ git config --global user.email "joserh201920@gmail.com"
```

4. SE HIZO EL PRIMER **GIT COMMIT - M** PARA GUARDAR LOS CAMBIOS REALIZADOS EN EL REPOSITORIO LOCAL Y AGREGARLE UN MENSAJE DESCRIPTIVO PARA IDENTIFICAR QUÉ CAMBIOS HICIMOS.

```
JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$ git commit -m "primer comit: Taller_semana1_2"
[master (root-commit) f6585ab] primer comit: Taller_semana1_2
4 files changed, 40 insertions(+)
create mode 100644 .classpath
create mode 100644 .project
create mode 100644 .settings/org.eclipse.core.resources.prefs
create mode 100644 .settings/org.eclipse.jdt.core.prefs

JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$ |
```

5. SE EJECUTÓ EL COMANDO **GIT LOG**, EL CUAL NOS PERMITE MOSTRAR EL HISTORIAL DE COMMITS DEL REPOSITORIO, PERMITIENDO VER LOS CAMBIOS GUARDADOS, QUIÉN LOS REALIZÓ Y CUÁNDO SE HICIERON.

```
JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$ git log
commit f6585ab7f8758dd851d244ab70daefcdc590a8f3 (HEAD -> master)
Author: grupo 10 <joserh201920@gmail.com>
Date: Tue May 26 20:55:15 2026 -0500

    primer comit: Taller_semana1_2

JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$ |
```

6. **GIT BRANCH** GRUPO-10 SIRVE PARA CREAR UNA NUEVA RAMA LLAMADA “GRUPO-10” DENTRO DEL REPOSITORIO, PERMITIENDO TRABAJAR EN CAMBIOS O VERSIONES SEPARADAS SIN AFECTAR LA RAMA PRINCIPAL.

```
JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$ git branch grupo-10

JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$ git checkout grupo-10
Switched to branch 'grupo-10'

JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (grupo-10)
$
```

7. **GIT ADD** SIRVE PARA AGREGAR O PREPARAR ARCHIVOS MODIFICADOS PARA QUE GIT LOS TOME EN CUENTA EN EL PRÓXIMO COMMIT, REALIZAMOS NUESTRO **SEGUNDO GIT COMMIT**

```
MINGW64:/c:/Users/JOSE/JRios-workspace/Taller_Semana1_2
JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (grupo-10)
$ git add .

JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (grupo-10)
$ git commit -m "mi segundo commit"
[grupo-10 7db8b20] mi segundo commit
13 files changed, 167 insertions(+)
create mode 100644 bin/Caso1.class
create mode 100644 bin/Caso2.class
create mode 100644 bin/Caso3.class
create mode 100644 bin/Caso4.class
create mode 100644 bin/Caso5.class
create mode 100644 bin/Contador.class
create mode 100644 bin/operacion.class
create mode 100644 bin/Persona.class
create mode 100644 src/Caso1.java
create mode 100644 src/Caso2.java
create mode 100644 src/Caso3.java
create mode 100644 src/Caso4.java
create mode 100644 src/Caso5.java

JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (grupo-10)
$
```

8. EJECUTAMOS **GIT CHECKOUT** PARA REGRESAR A NUESTRA RAMA PRINCIPAL **MASTER** PERMITIÉNDONOS TRABAJAR NUEVAMENTE

```
JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (grupo-10)
$ git checkout master
Switched to branch 'master'
```

9. EJECUTAMOS **GIT MERGE** PARA FUSIONAR LOS CAMBIOS DE LA RAMA GRUPO-10 CON LA RAMA ACTUAL INTEGRANDO SUS MODIFICACIONES EN UNA SOLA VERSIÓN DEL PROYECTO.

```
JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$ git merge grupo-10
Updating f6585ab..7db8b20
Fast-forward
 bin/Caso1.class      | Bin 0 -> 2387 bytes
 bin/Caso2.class      | Bin 0 -> 731 bytes
 bin/Caso3.class      | Bin 0 -> 1333 bytes
 bin/Caso4.class      | Bin 0 -> 1290 bytes
 bin/Caso5.class      | Bin 0 -> 2411 bytes
 bin/Contador.class   | Bin 0 -> 364 bytes
 bin/operacion.class  | Bin 0 -> 553 bytes
 bin/Persona.class    | Bin 0 -> 1087 bytes
 src/Caso1.java       | 49 ++++++
 src/Caso2.java       | 29 ++++++
 src/Caso3.java       | 26 ++++++
 src/Caso4.java       | 28 ++++++
 src/Caso5.java       | 35 ++++++
13 files changed, 167 insertions(+)
create mode 100644 bin/Caso1.class
create mode 100644 bin/Caso2.class
create mode 100644 bin/Caso3.class
create mode 100644 bin/Caso4.class
create mode 100644 bin/Caso5.class
create mode 100644 bin/Contador.class
create mode 100644 bin/operacion.class
create mode 100644 bin/Persona.class
create mode 100644 src/Caso1.java
create mode 100644 src/Caso2.java
create mode 100644 src/Caso3.java
create mode 100644 src/Caso4.java
create mode 100644 src/Caso5.java

JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$ |
```

- **CONEXIÓN CON LA NUBE (GITHUB)**

- **PASO 1:** `git remote add origin https://github.com/...`: Construye el puente de conexión. Le dice a tu Git local que la dirección remota de internet (en la nube de GitHub) se llamará origin.

Finalmente, para enviar tu repositorio local de la PC a internet:

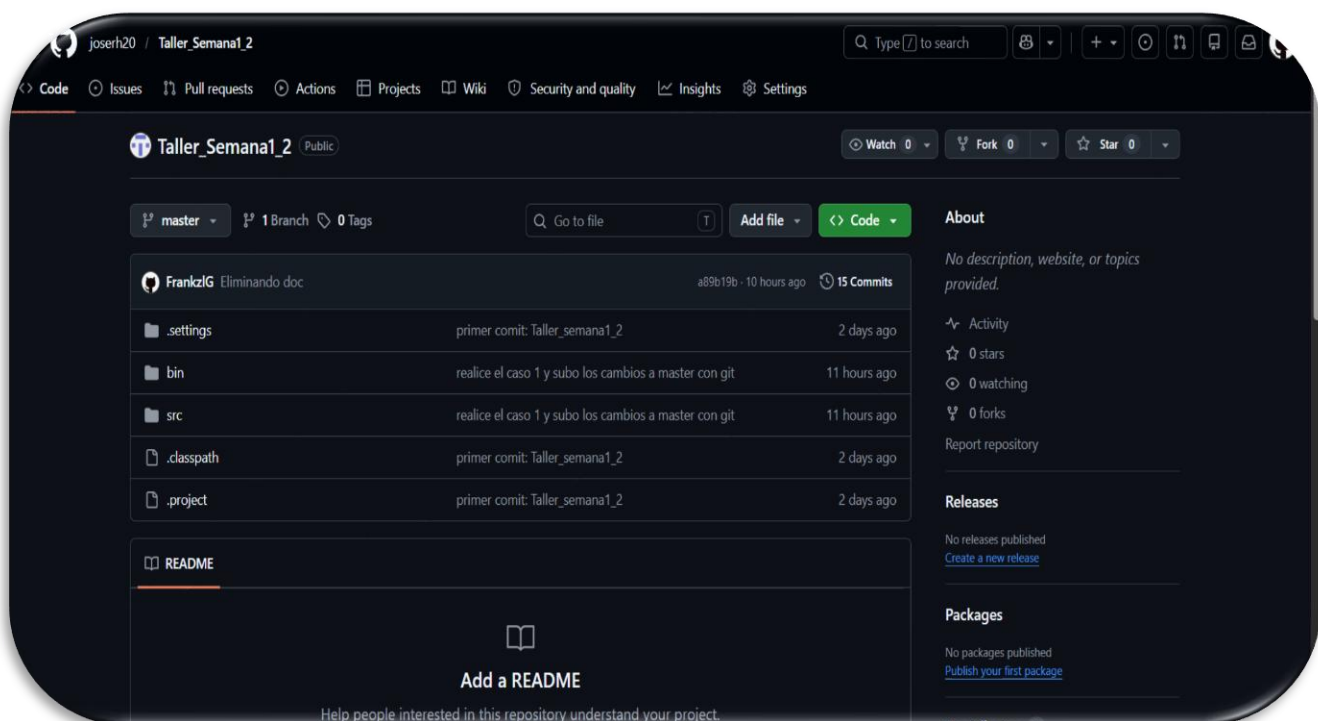
- **PASO 2:** `git push -u origin master`: Sube tus archivos a internet. Empuja todos los commits guardados en tu rama local master hacia el repositorio remoto en GitHub. El -u hace que recuerde la ruta para que las próximas veces solo tengas que escribir git push

```
JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$ git remote add origin https://github.com/joserh20/Taller_Semana1_2.git

JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
$ git push -u origin master
Enumerating objects: 24, done.
Counting objects: 100% (24/24), done.
Delta compression using up to 8 threads
Compressing objects: 100% (24/24), done.
Writing objects: 100% (24/24), 9.10 KiB | 665.00 KiB/s, done.
Total 24 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), done.
To https://github.com/joserh20/Taller_Semana1_2.git
 * [new branch]      master -> master
 * [branch 'master' set up to track 'origin/master'.

JOSE@LAPTOP-4IH75TU2 MINGW64 ~/JRios-workspace/Taller_Semana1_2 (master)
```

- **INTERFAZ DE GITHUB**



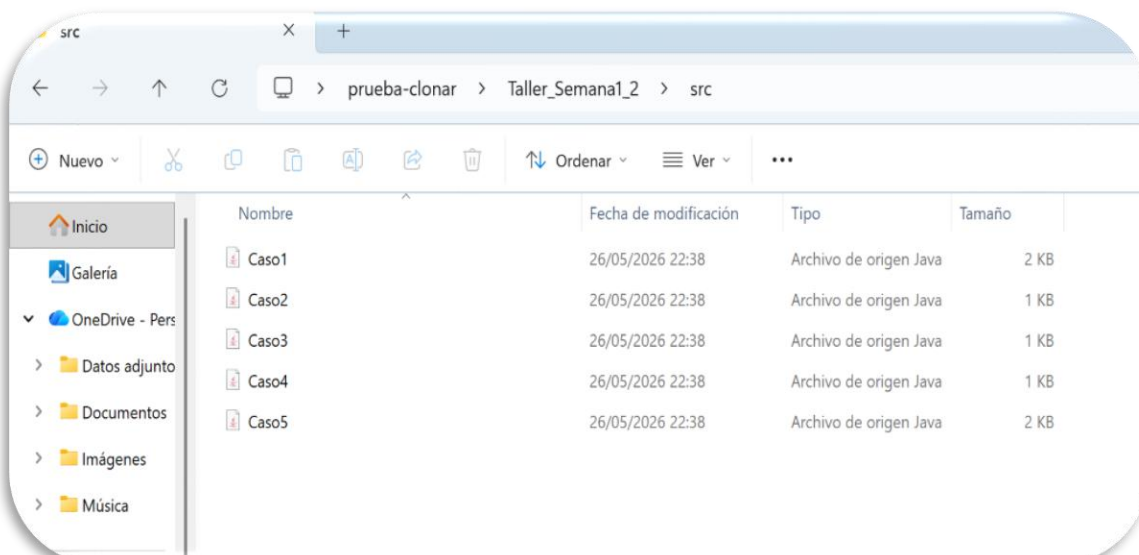
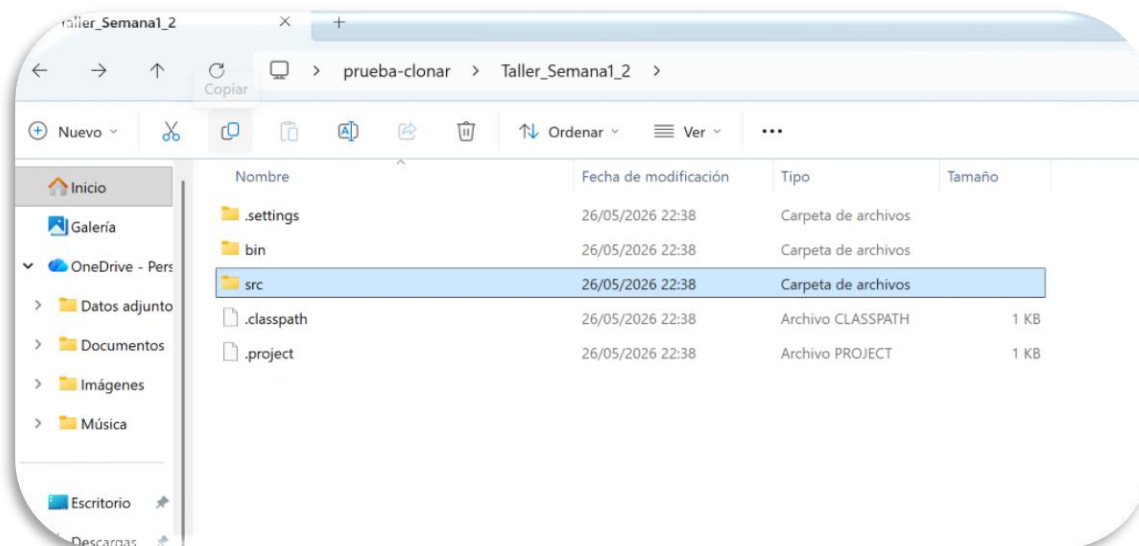
- **CLONANDO REPOSITORIO GITHUB**

```
MINGW64:/c/Users/FRANKLIN LOPEZ/Desktop/FRANKZLG
FRANKLIN LOPEZ@Frankz1G MINGW64 ~/Desktop (master)
$ cd FRANKZLG/

FRANKLIN LOPEZ@Frankz1G MINGW64 ~/Desktop/FRANKZLG (master)
$ ls

FRANKLIN LOPEZ@Frankz1G MINGW64 ~/Desktop/FRANKZLG (master)
$ git clone https://github.com/joserh20/Taller_Semana1_2.git
Cloning into 'Taller_Semana1_2'...
remote: Enumerating objects: 67, done.
remote: Counting objects: 100% (67/67), done.
remote: Compressing objects: 100% (45/45), done.
remote: Total 67 (delta 23), reused 54 (delta 17), pack-reused 0 (from 0)
Receiving objects: 100% (67/67), 786.84 KiB | 580.00 KiB/s, done.
Resolving deltas: 100% (23/23), done.

FRANKLIN LOPEZ@Frankz1G MINGW64 ~/Desktop/FRANKZLG (master)
```



¿Cuál es la diferencia entre un método normal y uno static?

Método Normal (de Instancia):

Pertenece a un objeto específico (una instancia). Es obligatorio crear primero el objeto utilizando la palabra `new` antes de poder usar su método.

Ejemplo: Si tienes una clase `Persona`, un método normal sería `caminar()`. No puedes simplemente decir "caminar" en el aire necesitas crear una persona específica para que realice la acción (`persona1.caminar()`).

Método Static (Estático o de Clase):

Le pertenece a la *clase* en general (al plano o plantilla), no a un objeto individual. No necesitas usar `new`. Lo llamas directamente usando el nombre de la clase. Son como herramientas de uso público.

Ejemplo: Las operaciones matemáticas. No necesitas crear un "objeto matemático" para sacar una raíz cuadrada. Simplemente llamas a la herramienta general usando la clase: `Math.sqrt(25)`.

¿En qué casos usarías sobrecarga de métodos?

La sobrecarga tener varios métodos con el mismo nombre pero que piden diferentes parámetros se utiliza cuando necesitas realizar exactamente la misma acción lógica, pero la información de entrada puede variar en cantidad o en tipo de dato.

Casos ideales para usarla:

Flexibilidad en los tipos de datos: Cuando quieres calcular algo, pero no sabes si te van a enviar números enteros (`int`) o números con decimales (`double`). Sobrecargas el método para que el programa acepte ambos sin lanzar un error y sin obligar al usuario a hacer conversiones manuales.

- `sumar(int a, int b)`
- `sumar(double a, double b)`

Datos opcionales (Valores por defecto): Cuando un proceso requiere varios datos, pero a veces el usuario no tiene todos.

- `registrarUsuario(String nombre, String correo, String telefono)`
- `registrarUsuario(String nombre, String correo)` *(Si usan este método, tú puedes programarlo para que asuma que no hay teléfono sin que el sistema falle).*

Facilitar la vida del programador: El mejor ejemplo que ya usas es `System.out.println()`. Es un método sobrecargado miles de veces internamente. Por eso funciona igual de bien si le pasas un texto "Hola", un número 10 o un cálculo `5 + 5`. Se llama igual, hace lo mismo (imprimir en pantalla), pero se adapta inteligentemente a lo que le des.

Información Adicional

1. Comandos de Rescate (Control de Daños)

- **Ctrl + C:** No es un comando escrito, pero es la combinación de teclas clave en la terminal para **detener a la fuerza cualquier proceso** que se haya quedado colgado o cargando infinitamente.
- **rm -rf .git:** Elimina por completo la carpeta oculta de Git. Lo usamos para **borrar el repositorio accidental** que se había creado en la ruta principal del usuario (C:\Users\JOSE) y así dejar esa carpeta limpia.

2. Configuración Inicial (Identidad)

Antes de guardar tu primer archivo, le indicamos a Git quién eres para que tus trabajos lleven tu firma en GitHub:

- **git config --global user.name "Tu Nombre":** Configura el nombre del autor de los commits.
- **git config --global user.email "tu@correo.com":** Configura el correo electrónico asociado a tu cuenta de GitHub.

3. El Ciclo de Guardado Local (El flujo de trabajo básico)

Una vez dentro de la carpeta correcta de Eclipse (Taller_Semana1_2), este es el motor principal para registrar tus avances:

- **git init:** **Inicializa un repositorio local vacío.** Crea la "cámara de seguridad" (carpeta oculta .git) dentro de la carpeta del proyecto para empezar a rastrear los cambios.
- **git status:** Te muestra el **estado actual de tus archivos**. Te dice en color rojo qué archivos han cambiado o son nuevos y todavía no se han guardado, y en verde los que ya están listos para guardarse.
- **git add .:** Agrega los archivos al "Staging Area" (área de preparación). El **punto (.)** es una instrucción que significa **"agrega absolutamente todo lo que esté en esta carpeta"**. Es el paso previo obligatorio antes de guardar.
- **git commit -m "mensaje":** **Guarda oficialmente los cambios en el historial local** de tu computadora con una etiqueta o mensaje descriptivo. (*Aquí corregimos el error de tipeo de commit y aprendimos a no anidar comillas dobles para que no se trabe la terminal*).
- **git log:** Muestra el **historial completo de commits**. Te enseña la lista de todos los guardados que has hecho, con el código único del commit, el autor, la fecha y la hora exacta.

4. Trabajo con Ramas (Manejo de Branches y Merges)

Para cumplir con los requisitos de trabajo en equipo y ramas de la rúbrica:

- **git branch grupo-10: Crea una nueva rama** llamada grupo-10. Las ramas permiten hacer experimentos o avanzar código por separado sin alterar la línea principal de trabajo.
- **git checkout grupo-10: Cambia de rama.** Te mueve de la rama principal a la rama especificada. Al ejecutarlo, la palabra azul de la terminal cambió a (grupo-10).
- **git checkout master:** Te regresa a la **rama principal** del proyecto (en tu terminal llamada master).
- **git merge grupo-10: Fusiona las ramas.** Estando parados en master, absorbe e integra todos los cambios y archivos que habías guardado en la rama grupo-10.

5. Conexión con la Nube (GitHub)

- **git remote add origin https://github.com/...:** Construye el puente de conexión. Le dice a tu Git local que la dirección remota de internet (en la nube de GitHub) se llamará origin.

Finalmente, para enviar tu repositorio local de la PC a internet:

- **git push -u origin master: Sube tus archivos a internet.** Empuja todos los commits guardados en tu rama local master hacia el repositorio remoto en GitHub. El -u hace que recuerde la ruta para que las próximas veces solo tengas que escribir git push.